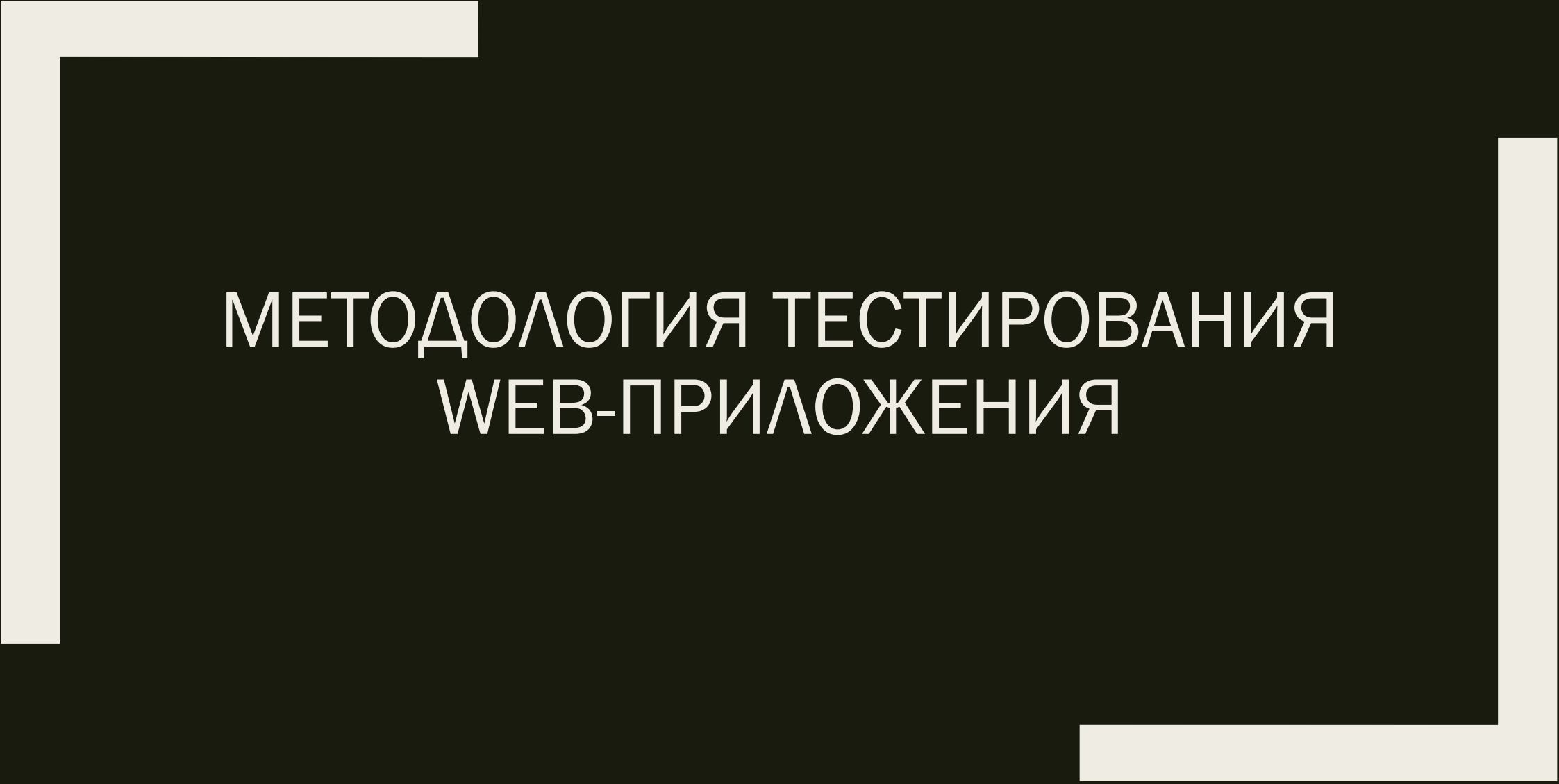




МЕТОДОЛОГИЯ ТЕСТИРОВАНИЯ WEB-ПРИЛОЖЕНИЯ

Особенности тестирования веб-приложений



Особенности тестирования веб-приложений

Доводилось ли вам тестировать веб-приложения? Практически любой специалист по тестированию программного обеспечения с опытом более года даст утвердительный ответ на этот вопрос, ведь существуют вполне объективные причины такого положения дел:

- на данный момент в сети Интернет действует более миллиарда сайтов, и пользуются ими более 3,5 млрд. людей по всему миру;
- в России более 70% взрослого населения являются интернет-пользователями, а общий оборот средств на российском рынке интернет-торговли за первое полугодие 2018 года вырос на 26% в сравнении с аналогичным периодом 2015 года и достиг 405 млрд. рублей.

Особенности тестирования веб-приложений

Этот процесс приводит к необходимости привлечения большого количества специалистов. То, что веб (в широком смысле) будет продолжать наращивать темпы своего развития, подтверждается и набирающим силу «мейнстримом»: всё «переезжает» в облака. **Облачные технологии становятся новой реальностью современного Интернета**: даже некогда привычные нам десктопные Word и Excel сегодня представлены в виде веб-альтернатив от Microsoft и Google. Исходя из сказанного, можно утверждать, что потребность в хороших инженерах по обеспечению качества, специализирующихся на веб-продуктах, будет только расти.

Клиент, сервер и база данных

Веб-приложение – это клиент-серверное приложение, в котором клиентом выступает браузер, а сервером – веб-сервер (в широком смысле). Основная часть приложения, как правило, находится на стороне веб-сервера, который обрабатывает полученные запросы в соответствии с бизнес-логикой продукта и формирует ответ, отправляемый пользователю. На этом этапе в работу включается браузер, именно он преобразовывает полученный ответ от сервера в графический интерфейс, понятный рядовому пользователю.

Клиент, сервер и база данных



Клиент, сервер и база данных

1. Клиент.

Как правило, **клиент – это браузер**, но встречаются и исключения (в тех случаях, когда один веб-сервер (BC1) выполняет запрос к другому (BC2), роль клиента играет веб-сервер BC1). В классической ситуации (когда роль клиента выполняет браузер) для того, чтобы пользователь увидел графический интерфейс приложения в окне браузера, последний **должен обработать полученный ответ веб-сервера, в котором будет содержаться информация, реализованная с применением HTML, CSS, JS** (самые используемые технологии). Именно эти технологии «дают понять» браузеру, как именно необходимо «отрисовать» все, что он получил в ответе.

Клиент, сервер и база данных

2. Сервер.

Веб-сервер – это сервер, принимающий HTTP-запросы от клиентов и выдающий им HTTP-ответы. Веб-сервером называют как программное обеспечение, выполняющее функции веб-сервера, так и непосредственно компьютер, на котором это программное обеспечение работает. Наиболее распространенными видами ПО веб-серверов являются Apache, IIS и NGINX. На веб-сервере функционирует тестируемое приложение, которое может быть реализовано с применением самых разнообразных языков программирования: PHP, Python, Ruby, Java, Perl и пр.

Клиент, сервер и база данных

3. База данных.

В классической теории речь идет о двух «сторонах» веб-приложения, однако, если внимательно посмотреть на весь процесс работы приложений, мы можем отметить, что в алгоритме работы веба незримо, но довольно активно принимает участие еще одна «сторона» – **база данных**. Фактически, **она не является частью веб-сервера, но большинство приложений просто не могут выполнять все возложенные на них функции без нее, так как именно в базе данных хранится вся динамическая информация приложения (учетные, пользовательские данные и пр).**

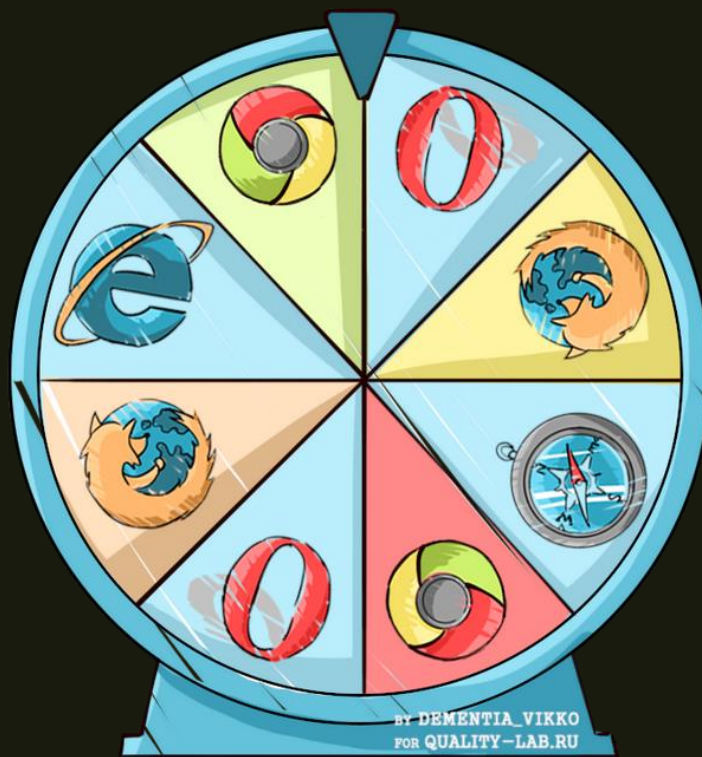
Клиент, сервер и база данных

База данных – довольно широкое понятие, которое используется не только в сфере информационных технологий, а также это – информационная модель, позволяющая упорядоченно хранить данные об объекте или группе объектов, обладающих набором свойств, которые можно категоризировать. Базы данных функционируют под управлением так называемых систем управления базами данных (далее – СУБД). Самыми популярными СУБД являются MySQL, MS SQL Server, PostgreSQL, Oracle (все – клиент-серверные).

Клиент, сервер и база данных

Также существуют встраиваемые и файл-серверные СУБД. Для общего развития отмечу лишь одну популярную встраиваемую СУБД – SQLite, которая используется в некоторых браузерах, Android API, Skype и других известных приложениях. Взаимодействие с перечисленными СУБД основано на специальном языке структурированных запросов – SQL.

Особенности архитектуры: клиент



Особенности архитектуры: клиент

Кроссбраузерность: разнообразие клиентов

Что мы понимаем под тестированием на кроссбраузерность? Это проверка на правильность (соответствие требованиям и стандартам) отображения и функционирования веб-приложения в разных браузерах и на разных операционных системах. В современном мире стандартизация принимает глобальные масштабы, а потому большинство популярных браузеров одинаково обрабатывают код. При этом необходимость в кроссбраузерном тестировании не исчезает, так как далеко не все проблемы решаются стандартизацией.

Особенности архитектуры: клиент

Перед началом работ рекомендуется выяснять у ответственных лиц необходимый для тестируемого ПО набор браузеров. Можно самим собрать статистику по целевой аудитории вашего продукта и ограничить кроссбраузерное тестирование набором наиболее популярных в этой аудитории браузеров. В условиях регрессионного тестирования, когда ограничены временные и человеческие ресурсы, также можно применять практику распределения браузеров: закрепляя за каждым тестировщиком определенный браузер, вы сможете «покрыть» большой процент кроссбраузерных дефектов. Этот метод имеет и свои минусы: он наиболее результативен в большой команде тестирования, но становится практически неприменимым при наличии одного тестировщика.

Особенности архитектуры: клиент

Что необходимо проверять при кроссбраузерном тестировании:

- функциональные возможности продукта, реализуемые на стороне клиента;
- правильность отображения элементов графики;
- шрифты и размеры текстовых символов;
- доступность и функциональность разнообразных форм, включая их интерактивность.

Особенности архитектуры: клиент

Отдельно рекомендуется не забывать о всякого рода валидаторах верстки, например, <https://validator.w3.org/>. Не стоит забывать, что иногда валидаторы обращают внимание на самые «мелочные мелочи», которые никто и никогда исправлять не будет. Если вы и заводите баг-репорты на подобного рода замечания, то удобнее будет собрать их в единый документ и прикрепить к репорту. К такого рода «мелочам» можно отнести всевозможные рекомендации, которые не имеют своего влияния на отображение и функционирование контента.

Особенности архитектуры: клиент

Веб-формы на стороне клиента

Одной из важных составляющих интернет-приложений являются **формы для заполнения, взаимодействие с которыми пользователь осуществляет с помощью все того же пристально рассмотренного нами клиента**. Однако, данные формы очень часто служат источником дефектов, которые, обосновавшись в «продакшене», могут принести большие финансовые и репутационные убытки компании.

Особенности архитектуры: клиент

Как же не пропустить дефекты в формах на продакшен? Рассмотрим несколько простых шагов:

1. Тщательно проверяем обязательность заполнения полей и наличие соответствующей маркировки у них.
2. Заполнив и отправив форму, убеждаемся в том, что с данными происходит именно то, что запланировано. Если данные должны быть внесены в базу данных, проверяем, корректно ли завершился процесс.
3. Используем чит-листы для тестирования форм.
4. Проверяем, выводятся ли понятные пользователю информационные сообщения о необходимости заполнения пустых полей после попытки отправить форму.

Особенности архитектуры: клиент

5. Обращаем пристальное внимание на реализацию экранирования символов в полях форм, являющихся потенциальным источником уязвимостей для приложения и пользователей. Экранирование должно осуществляться на уровне не только клиента, но и сервера, отключить который в клиенте довольно просто (например, с помощью специальных плагинов, снимающих все возможные ограничения в несколько кликов, таких как Web Developer Toolbar – Forms).

Особенности архитектуры: клиент

6. Убеждаемся, что после заполнения формы пользователю приходит подтверждающее письмо с указанием соответствующего отправителя, а само тело письма соответствует требованиям (в том числе и на работоспособность ссылок).
7. Используем вспомогательные специальные инструменты для тестирования форм (например, Web Developer Toolbar).

Особенности архитектуры: клиент

Текст, как основной источник информации при работе через клиент

Особая ценность сети Интернет заключается в том, что она является практически безграничным источником информации. Часть этой информации представлена в виде текстов, с которыми, опять же, пользователь взаимодействует посредством клиента. Большинство веб-ресурсов в том или ином объеме требуют проверки текстов на предмет отсутствия грамматических ошибок и опечаток.

Конечно, значимость этого тестирования не так велика по сравнению с функциональным направлением, но пренебрегать им не стоит.

Особенности архитектуры: сервер

Тестирование части веб-приложения, размещенной на веб-сервере, можно провести и минуя графический (клиентский) интерфейс, однако это требует от специалиста определенного уровня знаний и навыков технического характера, а также применения дополнительных инструментов. Рассмотрим веб-сервер с точки зрения нагрузочного и инсталляционного тестирования.

Особенности архитектуры: сервер

Инсталляция на веб-сервер

Прежде чем приступить к тестированию, необходимо установить (инсталлировать) веб-приложение на веб-сервер. Собственно, в этом есть сходство с проверкой десктоп-приложений, но существует и различие в нюансах, которые необходимо учесть и протестировать, особенно если это касается ПО, распространяемого для локальной инсталляции на веб-серверы пользователей.

Особенности архитектуры: сервер

В чем же заключаются эти нюансы?

1. Большая часть веб-приложений требует для инсталляции специфических знаний в администрировании ОС. Попробуйте установить приложение на нескольких веб-серверах. В оптимальном случае это будут самые популярные технологии среди ваших пользователей, для установления списка которых потребуется предварительное исследование.

2. Инсталлируя веб-приложение, обращайтесь внимание на то, действительно ли приложение устанавливается в указанную вами директорию, базу данных, использует выбранный вами префикс и соблюдает прочие конфигурационные моменты.

Особенности архитектуры: сервер

3. Убедитесь, что приложение можно установить, как из localhost, так и удаленно.

4. Если процесс инсталляции является интерактивным, и каждый выбор пользователя на определенном шаге влияет на последующие действия, то необходимо будет пройти все ветви, так как именно инсталляция является первым шагом пользователя в работе с вашим приложением.

5. Не забывайте о негативных тестах: проверки в условиях недостаточности ресурсов (места на диске, оперативной памяти) или привилегий, прерывание процесса установки во время активной его фазы (например, отправляя сервер в перезагрузку).

Особенности архитектуры: сервер

Нагрузочное тестирование

Нагрузочное тестирование имитирует работу с приложением определенного количества пользователей. Этот вид тестирования осуществляется при помощи специальных инструментов (например, jMeter), главная цель которых – определить профили нагрузки и искусственно создать для них нагрузку, выявляющую граничные возможности приложения (или сервера) в условиях работы с ним того или иного количества пользователей.

Особенности архитектуры: сервер

Полученная информация подвергается тщательному анализу с последующим выявлением «узких горлышек» и граничных программных и аппаратных возможностей, которые в дальнейшем используются для обеспечения стабильности веб-сервера и самого приложения, работающего на нем.

Особенности архитектуры: база данных

Еще одной ранее рассмотренной составляющей веб-приложений является база данных, в которой приложение хранит всю необходимую информацию. Для того, чтобы база данных служила достойным хранилищем информации для вашего приложения, при тестировании необходимо обращать внимание на следующие основные моменты:

1. Вносимая через интерфейс информация должна быть сохранена в базе данных в неизменном (первоначальном) виде.
2. Сохраненная в базе данных информация должна отображаться в любой части приложения одинаково (если иного не требует бизнес-логика приложения).

Особенности архитектуры: база данных

3. Названия таблиц и структура БД должны соответствовать проектной документации.

4. Нужно следить за тем, чтобы запросы не обрабатывались слишком долго, а количество соединений было достаточным. Мониторинг состояния БД – один из важных моментов тестирования.

5. Стоит учитывать, что удаление записи в БД не всегда сопровождается полным удалением сущности. Иногда используется так называемое «псевдоудаление», и нужно проверить, правильно ли оно выполняется.

Особенности архитектуры: база данных

6. Пустую БД на тестовом стенде рекомендуется либо заполнять сгенерированными случайными данными, либо снимать дампы с продакшена и после обфускации данных «заливать» их в тестируемую БД. Иногда квалификация тестировщика (или иная причина) не позволяет выполнить этот процесс самостоятельно: в таком случае рекомендуется обратиться за помощью к специалистам инфраструктуры или разработчикам.

Особенности архитектуры: запросы

Все составляющие веб-приложения должны взаимодействовать между собой, и происходит это благодаря HTTP(s). Без HTTP наша многосторонняя система не функционировала бы в принципе, так как HTTP – это протокол передачи данных, занимающий одно из основных мест в нашей клиент-серверной архитектуре.

Взаимодействие осуществляется через сообщения (запросы и ответы): на отправленный запрос от клиента должен прийти ответ сервера. **Классический запрос/ответ состоит из 3 составляющих:**

- стартовая строка;
- заголовок;
- тело сообщения.

Особенности архитектуры: запросы

При работе с ответами специалист по тестированию в первую очередь должен обращать внимание на методы и коды состояния, которые присутствуют в стартовой строке.

Самыми популярными методами являются GET, HEAD и POST:

1. Метод GET. Используется для запроса содержимого, размещенного на сервере (например, GET /path/resource?param1=value1¶m2=value2 HTTP/1.1).

2. Метод HEAD. По своей сути не отличается от вышеупомянутого метода, однако ответ сервера на такой запрос лишен «тела», а практическое применение ориентировано на облегченное использование с целью получения минимальной информации о сервере/продукте или его статусе.

Особенности архитектуры: запросы

3. Метод POST. Данный метод используется для передачи данных на сервер, однако его основа «прячется» в тело, что отличает его от GET. Во время публикации этой статьи, например, весь текст будет помещен в тело POST-запроса; после обработки его сервером на сайт будет добавлена статья.

Существуют и другие методы: PUT, DELETE, CONNECT, TRACE, PATCH и т. д.

Особенности архитектуры: запросы

То, что составляющие веб-приложения взаимодействуют между собой посредством HTTP, является хорошей новостью для специалиста по тестированию, так как это позволяет не просто отслеживать общение, вникая в логику работы и сверяя ее с техническими требованиями, но и влиять непосредственно на приложение, прикладывая свою руку к отправляемым запросам.

Классическими приложениями, которые можно использовать для генерации запросов, является Fiddler или Postman. Используя Fiddler, можно с легкостью отслеживать все запросы от клиента и ответы, просматривать их детали, а также вносить свои изменения и отправлять модифицированные запросы на сервер, оценивая поведение системы в таком случае.

Особенности архитектуры: запросы

На что обращать внимание в запросах?

1. Правильный ли метод используется для того или иного запроса? Если вы отправляете сообщение, а на сервер уходит только GET-запрос, то что-то здесь явно не так.

2. Казалось бы, клик по одной и той же функциональной клавише генерирует один и тот же запрос. Но есть прецеденты, когда клик по кнопке приводил к ситуации, в которой JavaScript переписывал запрос рядом находящейся кнопки, таким образом меняя ее предназначение.

Особенности архитектуры: запросы

3. Вникайте в отправляемые запросы. В них довольно легко разобраться, особенно если это GET – они логически понятны даже рядовому пользователю. Анализ запросов – это возможность обнаружить спрятавшийся дефект гораздо быстрее, чем осуществляя его поиск в интерфейсе.

4. Мониторьте трафик на предмет запросов на другие (не ваши) сервера. Пример из жизни: фронтэнд сайта делал фриланс-разработчик, по завершению работы которого мы принялись за тестирование. Я имею привычку мониторить весь трафик тестируемого приложения: это позволило обнаружить, что вышеупомянутый разработчик без зазрения совести спрятал в «фронт» сайта запросы, которые работали на благо его личного интернет-магазина.

5. Мониторя трафик, внимательно следите за кодами состояний.

Особенности архитектуры: коды

Коды состояния глазами адепта качества

Каждый ответ на любой запрос несет в себе массу полезной для разработчиков и специалистов по тестированию ПО информации. Одним из источников такой информации может стать **код состояния: по этому коду клиент узнаёт о результатах его запроса и определяет, какие действия ему предпринять дальше**. Набор кодов состояния является стандартом, он описан в соответствующих документах RFC. Каждый раз, когда вы создаете баг-репорт о неработающих ссылках, иных элементах веб-страницы или системных ошибках, обязательно указывайте ответы сервера: они сэкономят время разработчику при определении причин дефекта.

Особенности архитектуры: коды

Все коды можно поделить на группы (сотые, двухсотые, трехсотые, четырехсотые и пятисотые) каждая группа-«сотня» несет свой тип информации.

Код	Тип информации кода	Назначение
1xx	Информирующие	Информирование о передаче сообщения. Сами сообщения от сервера содержат только стартовую строку ответа и, если требуется, несколько специфичных для ответа полей заголовка.
2xx	Сообщающие об успешной операции	Информирование об успешной обработке запроса клиента. Самым классическим и распространенным является «200 OK».
3xx	Перенаправляющие	Сообщает, что для успешного выполнения операции необходимо сделать другой запрос. Из данного типа пять кодов: 301, 302, 303, 305 и 307, относятся непосредственно к перенаправлению. Адрес, по которому клиенту следует произвести запрос, сервер указывает в заголовке Location.
4xx	Ошибка на стороне клиента (запрос)	Указание на ошибки со стороны клиента. При использовании всех методов, кроме HEAD, сервер возвращает пояснения причин.
5xx	Ошибка на стороне сервера (ответ)	Говорит об ошибках выполнения операции по вине веб-сервера.

Особенности архитектуры

Важно помнить, что тестирование ПО ставит перед каждым вступившим в стройные ряды сферы обеспечения качества ПО такие задачи, которые практически невозможно решать однозначно и по четкому алгоритму. Тестирование – это философия, творчество, полет мысли, основанный на четких и безоговорочных технических аспектах. Пожалуй, только тестировщикам приходится одновременно быть разработчиком, системным аналитиком, пользователем, заказчиком (для которого первоочередной интерес – финансовый или иной успех продукта), специалистом в предметной области и даже DBA

Особенности архитектуры

Каждый из этих навыков тестировщик должен развивать постоянно, и тогда он сможет избежать многих возможных проблем и остаться востребованным высококвалифицированным профессионалом. Остановка в развитии для специалиста из сферы обеспечения качества фактически приравнивается к профессиональной смерти (как минимум, «клинической» в том случае, если стремление к новым знаниям со временем возвращается).